

Unit-2: Problems, State Space Search & Heuristic Search Techniques

Main Topic	Subtopics	Key Concepts to Understand
2.1 Defining the Problem as State Space Search	State space, Initial state, Goal state, Operators, Path, Solution path	How AI models problems as states and transitions
2.2 Production Systems	Definition of production systems, Components of production systems, Rule-based representation	Knowledge expressed using IF-THEN rules
2.3 Production System Characteristics	Monotonic production system, Non-monotonic production system, Partially commutative production system	Behavior of production systems during problem solving
2.4 Issues in Designing Search Programs	Search space size, Efficiency, Completeness, Optimality	Challenges in developing AI search algorithms
2.5 Generate and Test	Concept, algorithm steps, advantages, limitations	Basic trial-and-error search technique
2.6 Hill Climbing	Simple hill climbing, Steepest ascent hill climbing, Problems (local maxima, plateau, ridge)	Local search technique
2.7 Best-First Search	Evaluation function, Greedy search, Priority queue	Uses heuristic information to guide search
2.8 Problem Reduction	Problem decomposition, AND-OR graphs	Breaking complex problems into smaller subproblems
2.9 Constraint Satisfaction Problems (CSP)	Variables, domains, constraints, constraint propagation	Solving problems with restrictions
2.10 Means-Ends Analysis	Goal comparison, difference reduction, operator selection	Problem solving by reducing difference between current and goal state

2.1 Defining the Problem as State Space Search

Many AI problems are solved using **State Space Search**. The idea is to represent a problem as a **collection of states** and then search through those states to find a path from the **initial state to the goal state**.

Instead of solving the problem directly, AI **explores possible situations (states)** until it reaches the desired solution.

Basic Terms Related to State Space Search

Before understanding state space search, several important terms must be clearly defined.

Term	Meaning
State	Representation of a particular situation
State Space	All possible states of a problem
Initial State	Starting condition of the problem
Goal State	Desired final condition
Operators	Actions that change states
Path	Sequence of states from start
Solution Path	Correct path leading to goal

1. State

Meaning : A **state** represents the **current situation or configuration of a problem**.

Each state describes **how the system looks at a particular moment**.

Example

Consider the **8-puzzle problem**.

Initial puzzle:

1 2 3

4 5 6

7 _ 8

This arrangement is a **state**.

If the blank moves:

1 2 3

4 5 6

7 8 _

This becomes another state.

Each arrangement of tiles is a **different state**.

Key Idea

State = **snapshot of the problem at a given moment**

2. State Space

Meaning: The **state space** is the **set of all possible states that can occur in a problem**.

It includes:

- initial state
- intermediate states
- goal states

Example: For the **8-puzzle problem**, all possible tile arrangements form the **state space**.

Total possible states:

181,440 possible configurations.

Example: Chess

Every possible chess board configuration represents a state.

Total possible states:

$\approx 10^{120}$ positions.

This enormous number makes search difficult.

Visualization

State space is often represented as a **graph or tree**.

Initial State

|

v

State1

/ \

State2 State3

|

Goal State

Each node represents a state.

3. Initial State

Meaning : The **initial state** is the **starting point of the problem**.
It represents the situation before any actions are performed.

Example

In **maze solving**:

Initial state = starting position in the maze.

Example

In **8-puzzle**:

1 2 3

4 5 6

_ 7 8

This is the starting state.

4. Goal State

Meaning: The **goal state** represents the **desired solution or final state of the problem**.
All algorithms search through the state space until they reach this state.

Example

Goal state of 8-puzzle:

1 2 3

4 5 6

7 8 _

When this configuration is reached, the problem is solved.

5. Operators

Meaning: Operators are actions that change one state into another state.
They define how the system moves through the state space.

Example

In **8-puzzle**, operators are:

- move blank up
- move blank down
- move blank left
- move blank right

Each operator produces a new state.

Example

Robot navigation:

Operators may include:

- move forward
- turn left
- turn right

6. Path

Meaning: A path is a sequence of states connected by operators.
It represents the steps taken to move from one state to another.

Example

8-puzzle path:

State1 → State2 → State3 → State4

Each step represents an operator applied to the previous state.

Path Cost: Sometimes each step has a **cost**.

Example:

Action	Cost
Move tile	1

Total path cost = sum of all step costs.

Search algorithms often try to **minimize path cost**.

7. Solution Path

Meaning: The **solution path** is the path that leads from the **initial state to the goal state**.

It represents the **correct sequence of actions required to solve the problem**.

Example

8-puzzle solution path:

Initial State



State1



State2



Goal State

This sequence of moves solves the puzzle.

Example of State Space Search

Problem: Maze Navigation

Component	Example
Initial State	entrance of maze
Goal State	exit of maze
Operators	move up, down, left, right
State Space	all positions in maze
Solution Path	correct route to exit

Representation of State Space

State space problems are often represented using **graphs**.

Graph components:

Component	Meaning
Nodes	states
Edges	operators
Path	sequence of nodes

Why State Space Representation is Important

State space representation helps AI systems:

- systematically explore possible solutions
- apply search algorithms

- evaluate different paths
- find optimal solutions

Most classical AI algorithms work on **state space models**.

Examples:

- Breadth First Search
- Depth First Search
- A* Algorithm
- Best First Search

Key Summary

Concept	Meaning
State	current situation
State Space	all possible states
Initial State	starting point
Goal State	desired solution
Operators	actions changing states
Path	sequence of states
Solution Path	correct path to goal

These concepts form the **foundation of AI search algorithms**.

2.2 Production Systems

A **Production System** is a rule-based problem-solving model widely used in Artificial Intelligence. It represents knowledge using **rules** and applies these rules to transform the current state of a problem into a new state until the **goal state** is reached.

Production systems are commonly used in:

- Expert systems
- Decision support systems
- Automated reasoning systems

1. Definition of Production System

A **Production System** is a system that uses a set of **production rules (IF-THEN rules)** to solve problems by applying rules to the current state of the system.

Each rule describes **what action should be taken when a particular condition is true**.

Formal Definition

A production system consists of:

- a **set of rules**
- a **working memory**
- a **control strategy**

These components work together to perform problem solving.

Basic Idea of Production System

Problem solving occurs through repeated steps:

1. Check which rules can be applied
2. Select one rule
3. Apply the rule to change the current state
4. Continue until the goal state is reached

This cycle is known as the **recognize-act cycle**.

2. Components of Production System

A production system has three main components.

Component	Description
Rule Base (Production Rules)	Set of IF-THEN rules representing knowledge
Working Memory	Stores current facts or state of the system
Control Strategy	Determines which rule should be applied

1. Rule Base (Production Rules)

The **rule base** contains a set of production rules.

Each rule consists of:

IF condition THEN action

The condition checks whether a rule is applicable.

The action changes the system state.

Example

Rule for temperature:

IF temperature > 38°C

THEN patient has fever

Here:

Part	Meaning
IF condition	temperature > 38
THEN action	patient has fever

2. Working Memory

Working memory stores the **current information or state of the system**.

It contains facts that are used by the rules.

Example

Working memory may contain facts such as:

temperature = 39

patient has headache

patient has cough

These facts help determine which rules can be applied.

3. Control Strategy

The control strategy decides:

- which rule should be applied
- in what order rules are applied

This is important when **multiple rules are applicable**.

Common strategies include:

Strategy	Description
First applicable rule	choose first matching rule
Priority-based selection	choose highest priority rule
Specificity strategy	choose most specific rule

Rule-Based Representation

In production systems, knowledge is represented using **rules**.

Rules describe **relationships between conditions and actions**.

This is known as **Rule-Based Representation**.

Structure of Production Rule

General form:

IF condition

THEN action

The condition part checks facts.

The action part modifies the system state.

Example

Traffic rule:

IF traffic light = RED

THEN stop vehicle

Medical rule:

IF patient has fever AND cough

THEN possible infection

Knowledge Expressed Using IF-THEN Rules

In production systems, knowledge is expressed as a collection of rules.

These rules capture **expert knowledge**.

Example: Medical Expert System

Knowledge base:

IF patient has fever

THEN check infection

IF infection detected

THEN prescribe medicine

This system mimics a doctor's reasoning.

Operation of Production System

The production system works in cycles.

Steps:

1. Match rules with current facts
2. Select one rule
3. Execute the rule
4. Update working memory

This cycle repeats until the goal is achieved.

Recognize-Act Cycle

The basic operation is called the **Recognize-Act Cycle**.

Steps involved:

Step	Description
Recognize	find rules whose conditions match current state
Select	choose one rule
Act	apply rule and update system state

Example of Production System

Consider a simple **number transformation problem**.

Goal: convert number 2 to 10.

Rules:

R1: IF number < 10 THEN add 2

R2: IF number > 10 THEN subtract 1

Working memory:

number = 2

Execution:

2 → 4 → 6 → 8 → 10

Goal achieved.

Conflict Resolution

Sometimes **multiple rules match at the same time**.

The system must decide which rule to apply.

This situation is called **conflict**.

The method used to resolve it is called **conflict resolution**.

Example

Working memory:

temperature = 39

cough = yes

Rules:

Rule1: IF fever THEN infection

Rule2: IF cough THEN cold

Both rules match.

The system must choose one rule.

Advantages of Production Systems

Advantage	Explanation
Simple knowledge representation	easy to express rules
Modular structure	rules can be added or removed
Easy debugging	each rule independent
Natural reasoning	similar to human reasoning

Limitations of Production Systems

Limitation	Explanation
Large rule base	system becomes complex
Conflict resolution issues	many rules may match
Slow processing	rule matching may take time

Applications of Production Systems

Production systems are used in many AI applications.

Application	Example
Expert systems	medical diagnosis
Game playing	rule-based strategies
Natural language processing	grammar rules
Automated planning	decision systems

Example: Expert System Architecture

Knowledge Base (Rules)



Working Memory (Facts)



Inference Engine



Decision / Action

The **inference engine** applies production rules to solve problems.

Summary

Concept	Meaning
Production System	rule-based problem solving model
Production Rules	IF-THEN rules
Working Memory	stores current facts
Control Strategy	decides rule selection
Recognize-Act Cycle	process of applying rules

Production systems are one of the **fundamental approaches used in classical Artificial Intelligence** for knowledge representation and reasoning.

Production System Characteristics

We will study:

- Monotonic Production System
- Non-Monotonic Production System
- Partially Commutative Production System

These describe **how production systems behave during problem solving**.

Production System Characteristics : Production systems can behave differently depending on how rules affect the system state. These behaviors are described by **production system characteristics**. These characteristics explain **how applying one rule affects other rules and states** in the system.

The three main types are:

1. Monotonic Production System
2. Non-Monotonic Production System
3. Partially Commutative Production System

Overview of Production System Characteristics

Type	Main Idea	Effect on Rules
Monotonic Production System	Applying a rule never prevents other rules from being applied	Rules remain valid
Non-Monotonic Production System	Applying a rule may disable other rules	Rules may become invalid
Partially Commutative Production System	Order of rule application does not affect final result	Same result possible

1. Monotonic Production System

Meaning: A **Monotonic Production System** is one in which **applying a rule does not prevent other rules from being applied later**.

In other words, once a rule becomes applicable, it **remains applicable throughout the problem-solving process**.

The knowledge base only **adds new information but does not remove existing information**.

Key Characteristics

Property	Description
Information only increases	new facts are added
No information removal	previous facts remain valid
Rules remain applicable	rule execution does not disable others

Example

Working memory contains: AB

Rules:

R1: IF A THEN C

R2: IF B THEN D

Execution:

A, B → apply R1 → add C

A, B, C → apply R2 → add D

Both rules can be applied without interfering with each other.

Real Example

Mathematical reasoning.

Example:

IF $x = 2$ THEN $x^2 = 4$

IF $x^2 = 4$ THEN $x^2 + 2 = 6$

Adding new facts does not invalidate previous rules.

Key Idea : Information **only grows**, and rules do not interfere.

2. Non-Monotonic Production System

Meaning : A **Non-Monotonic Production System** is one in which **applying a rule may remove or change information, making other rules invalid**.

In this system, applying one rule can **prevent other rules from being used later**.

Information in working memory may be **modified or removed**.

Key Characteristics

Property	Description
Information may change	facts can be modified
Rules may become invalid	rule applicability may change
Knowledge may decrease	facts can be removed

Example

Working memory: Bird(Tweety)

Rules:

R1: IF Bird(x) THEN x can fly

R2: IF Penguin(x) THEN x cannot fly

Initially:

Bird(Tweety) → Tweety can fly

Later information appears: Penguin(Tweety)

Now the conclusion changes: Tweety cannot fly

Earlier conclusion becomes invalid.

Real Example

Medical diagnosis.

Initial assumption: IF fever THEN infection
Later tests show: IF viral infection THEN not bacterial infection
The diagnosis changes when new information arrives.

Key Idea : Knowledge may **change or be revised**.

3. Partially Commutative Production System

Meaning : A **Partially Commutative Production System** is one in which **the order of rule application does not affect the final result**.

Different rule sequences may be used, but they eventually produce the **same goal state**.

Key Characteristics

Property	Description
Rule order may vary	rules applied in different orders
Same final result	final solution remains unchanged
Flexible search paths	multiple valid paths

Example

Working memory: $x = 2$

Rules: R1: multiply by 2

R2: add 4

Two possible paths:

Path 1:

$$2 \times 2 = 4$$

$$4 + 4 = 8$$

Path 2:

$$2 + 4 = 6$$

$$6 \times 2 = 12$$

Here results differ, so not commutative.

But if operations produce same final state, system becomes partially commutative.

Example:

R1: add 2

R2: add 3

Paths:

$$2 + 2 + 3 = 7$$

$$2 + 3 + 2 = 7$$

Order does not matter.

Real Example

Cooking recipe.

Steps:

1. Cut vegetables
2. Wash vegetables

Order does not affect final outcome.

Comparison of Production System Characteristics

Feature	Monotonic	Non-Monotonic	Partially Commutative
Information change	Only increases	May change or decrease	May change
Rule validity	Always remains valid	May become invalid	Remains valid

Rule order importance	Usually fixed	Important	Order may vary
Stability	High	Moderate	Moderate

Importance in AI

Understanding these characteristics helps AI designers:

- build better reasoning systems
- design efficient search algorithms
- manage knowledge updates correctly

These properties are especially important in:

- expert systems
- reasoning systems
- knowledge-based AI systems

Summary

Type	Main Idea
Monotonic System	rules do not invalidate each other
Non-Monotonic System	new knowledge may change previous conclusions
Partially Commutative System	rule order does not affect final result

These characteristics describe **how production systems behave when rules are applied during problem solving.**

Types of Search Techniques in AI

Category	Description
Uninformed Search (Blind Search)	No additional knowledge, only uses problem definition
Informed Search (Heuristic Search)	Uses extra knowledge (heuristics) to guide search

1. Uninformed Search (Blind Search)

Meaning: Uninformed search methods **do not use any domain knowledge**. They only use:

- initial state
- goal state
- possible actions

They explore the search space **without guidance**.

Key Idea: Search without knowing which path is better

Types of Uninformed Search

Algorithm	Description
Breadth-First Search (BFS)	explores level by level
Depth-First Search (DFS)	explores deep paths first
Depth-Limited Search	DFS with depth restriction
Iterative Deepening	repeated DFS with increasing depth
Uniform Cost Search	considers path cost

Example

Maze problem:

- AI tries all possible paths
- no idea which path leads to goal faster

Advantages

Advantage	Explanation
Simple	easy to implement
Complete (BFS)	finds solution if exists

Limitations

Limitation	Explanation
Slow	explores many unnecessary nodes
Inefficient	no direction toward goal

2. Informed Search (Heuristic Search)

Meaning: Informed search uses **extra knowledge (heuristics)** to guide the search toward the goal. A heuristic is a **function that estimates how close a state is to the goal**.

Key Idea

Search with guidance (intelligent search)

Heuristic Function

$h(n)$ = estimated cost from node n to goal

Types of Informed Search

Algorithm	Description
Greedy Best-First Search	chooses node closest to goal
A* Search	combines cost + heuristic
Hill Climbing	moves toward better states
Beam Search	limits number of nodes explored

Example

GPS navigation:

- uses distance to destination
- chooses shortest path

Advantages

Advantage	Explanation
Efficient	explores fewer nodes
Faster	guided search
Optimal (A*)	finds best solution

Limitations

Limitation	Explanation
Requires heuristic	needs extra knowledge
May be incorrect	bad heuristic → wrong path

Difference Between Informed and Uninformed Search

Feature	Uninformed Search	Informed Search
Knowledge	no extra knowledge	uses heuristic
Efficiency	low	high
Speed	slow	fast

Example	BFS, DFS	A*, Greedy
---------	----------	------------

Real-World Understanding

Scenario	Search Type
Finding path blindly	Uninformed
Google Maps navigation	Informed

2.4 Issues in the Design of Search Programs

Key ideas include:

- search space size
- completeness
- optimality
- efficiency
- computational complexity.

2.4 Issues in Designing Search Programs : Search algorithms are used in AI to **find solutions by exploring the state space**. However, designing effective search programs is challenging because many problems have **very large numbers of possible states**.

Therefore, while designing search algorithms, several important issues must be considered.

The main issues are:

1. Search Space Size
2. Efficiency
3. Completeness
4. Optimality

These factors determine **how well a search algorithm performs**.

Overview of Issues in Search Programs

Issue	Meaning	Importance
Search Space Size	Total number of possible states	Determines complexity
Efficiency	Time and memory used by algorithm	Affects performance
Completeness	Ability to find a solution if it exists	Ensures reliability
Optimality	Ability to find the best solution	Ensures quality

1. Search Space Size

Meaning: The **search space** is the set of all possible states that can be reached from the initial state.

The **size of the search space** refers to the **total number of states the algorithm may need to explore**.

Why Search Space Size is Important

Many AI problems have extremely large search spaces.

Example:

Problem	Approximate Search Space
Tic-Tac-Toe	255,168 states
Chess	about 10^{120} states
Traveling Salesman Problem	factorial growth

Such enormous spaces make **exhaustive search impossible**.

Example

Consider a chess game.

At each move there are about **35 possible moves**.

If a game lasts 40 moves:

Possible combinations:

$$35^{40}$$

This number is extremely large.

This phenomenon is called **combinatorial explosion**.

Solutions to Handle Large Search Space

AI uses several methods:

- heuristic search
- pruning techniques
- problem decomposition
- constraint satisfaction

These techniques help reduce the number of states explored.

2. Efficiency

Meaning : Efficiency refers to **how quickly a search algorithm finds a solution while using minimal computational resources**.

Efficiency mainly depends on:

- time complexity
- memory usage

Time Efficiency

Time efficiency measures **how long the algorithm takes to find a solution**.

Example:

Algorithm	Time Taken
Algorithm A	2 seconds
Algorithm B	10 seconds

Algorithm A is more efficient.

Memory Efficiency : Memory efficiency measures **how much memory the algorithm uses**.

Some algorithms store large numbers of states in memory.

Example: Breadth First Search stores many nodes in memory.

Importance of Efficiency

AI applications often require **real-time responses**.

Examples:

- autonomous vehicles
- robotic control systems
- online recommendation systems

Efficient algorithms ensure quick decisions.

3. Completeness

Meaning : Completeness refers to **whether a search algorithm is guaranteed to find a solution if a solution exists**.

A complete algorithm will eventually find the goal state.

Example : Breadth First Search (BFS)

If the goal exists in the state space, BFS will find it.

Therefore BFS is **complete**.

Example of Incomplete Algorithm

Depth First Search (DFS) may fail if:

- the search space is infinite
- the algorithm gets stuck exploring a deep branch

Thus DFS is **not always complete**.

Importance of Completeness

Completeness ensures:

- reliability of AI systems
- guaranteed solution discovery

This is very important in critical applications.

Example:

- medical diagnosis systems
- automated planning systems

4. Optimality

Meaning: Optimality refers to **whether the search algorithm finds the best possible solution**.

The best solution usually means:

- lowest cost
- shortest path
- minimum time

Example

Consider route planning.

Possible paths:

Path	Distance
Path A	10 km
Path B	7 km

The optimal solution is **Path B** because it has the shortest distance.

Optimal Search Algorithms

Some algorithms guarantee optimal solutions.

Examples:

Algorithm	Optimality
Breadth First Search	Optimal for equal cost
Uniform Cost Search	Optimal
A* Search	Optimal (with proper heuristic)

Non-Optimal Algorithms

Some algorithms do not guarantee optimal solutions.

Example: Hill Climbing may get stuck at **local maxima**.

Relationship Between These Issues

These four factors often conflict with each other.

Factor	Effect
Large search space	reduces efficiency
High efficiency	may sacrifice optimality

Complete algorithms	may use more memory
Optimal algorithms	may take longer time

Designing search algorithms requires balancing these factors.

Example: Maze Solving

Issue	Explanation
Search space size	all possible maze positions
Efficiency	how fast the exit is found
Completeness	algorithm must find exit if exists
Optimality	shortest path to exit

Summary

Issue	Description
Search Space Size	number of possible states
Efficiency	time and memory usage
Completeness	ability to always find solution
Optimality	ability to find best solution

These issues help AI designers **evaluate and improve search algorithms**.

Breadth-First Search (BFS)

Breadth-First Search (BFS) is an **uninformed (blind) search algorithm** used in AI to explore a state space **level by level**.

It starts from the **initial node** and explores all its neighboring nodes first, then moves to the next level.

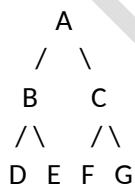
Overview

Concept	Meaning
BFS	explores nodes level by level
Search Type	Uninformed search
Data Structure	Queue (FIFO)
Goal	find shortest path in terms of number of steps

1. Basic Idea of BFS

Start → explore all neighbors → then next level → until goal found
BFS expands nodes in **increasing order of depth**.

Example Tree



BFS Traversal Order:

A → B → C → D → E → F → G

2. How BFS Works

BFS uses a **queue (FIFO: First In First Out)**.

Steps of BFS

Step	Description
1	start with initial node
2	add it to queue
3	remove node from queue
4	check if it is goal
5	add its children to queue
6	repeat until goal found

Example

Find node G:

Queue = [A]

→ visit A

Queue = [B, C]

→ visit B

Queue = [C, D, E]

→ visit C

Queue = [D, E, F, G]

→ visit G (goal found)

3. BFS Algorithm (Simple)

BFS(start):

1. create empty queue
2. add start node to queue
3. while queue is not empty:
 node = remove from queue

 if node is goal:
 return solution

 add all children of node to queue

4. Properties of BFS

Property	Value
Completeness	Yes (if solution exists)
Optimality	Yes (shortest path)
Time Complexity	$O(b^d)$
Space Complexity	$O(b^d)$

Where:

- **b** = branching factor
- **d** = depth of solution

5. Advantages of BFS

Advantage	Explanation
Finds shortest path	optimal solution

Complete	guaranteed to find solution
Simple implementation	easy to understand

6. Limitations of BFS

Limitation	Explanation
High memory usage	stores all nodes
Slow for large trees	explores many nodes
Not suitable for deep trees	inefficient

7. Applications of BFS

Application	Example
Pathfinding	shortest path in graphs
Web crawling	exploring web pages
Social networks	friend suggestions
AI search problems	state space search

8. BFS vs DFS (Quick Comparison)

Feature	BFS	DFS
Search Strategy	level-wise	depth-wise
Data Structure	queue	stack
Optimal	Yes	No
Memory	High	Low

Real-Life Example

Google Maps (shortest route):

- BFS ensures **minimum number of steps (roads)**
- finds shortest path in unweighted graph

Summary

Concept	Meaning
BFS	level-by-level search
Queue	FIFO structure
Optimal	finds shortest path
Use	pathfinding and graph traversal

BFS is one of the most important **basic search algorithms in AI**, forming the foundation for more advanced techniques.

Depth-First Search (DFS)

Depth-First Search (DFS) is an **uninformed (blind) search algorithm** that explores a search space by going **as deep as possible along one branch before backtracking**.

Instead of exploring level by level (like BFS), DFS **dives deep into the tree first** and only explores other branches when necessary.

Overview

Concept	Meaning
DFS	explores deepest nodes first
Search Type	Uninformed search

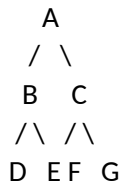
Data Structure	Stack (LIFO) or recursion
Goal	reach goal by deep exploration

1. Basic Idea of DFS

Start → go deep → reach leaf → backtrack → explore next branch

DFS follows a **depth-first strategy**.

Example Tree



DFS Traversal Order:

A → B → D → E → C → F → G

2. How DFS Works

DFS uses a **stack (LIFO: Last In First Out)**.

It explores the **most recently discovered node first**.

Steps of DFS

Step	Description
1	start with initial node
2	push node into stack
3	pop node from stack
4	check if goal
5	push its children into stack
6	repeat until goal found

Example

Find node G:

Stack = [A]

→ visit A

Stack = [B, C]

→ visit C

Stack = [B, F, G]

→ visit G (goal found)

(Note: order depends on child insertion)

3. DFS Algorithm (Simple)

DFS(node):

1. if node is goal:
 return solution
2. mark node as visited
3. for each child of node:
 DFS(child)

4. Properties of DFS

Property	Value
Completeness	No (may fail in infinite depth)
Optimality	No (not shortest path)
Time Complexity	$O(b^d)$
Space Complexity	$O(b \times d)$

Where:

- **b** = branching factor
- **d** = depth

5. Advantages of DFS

Advantage	Explanation
Low memory usage	stores fewer nodes
Simple implementation	recursion or stack
Fast in deep solutions	finds deep goal quickly

6. Limitations of DFS

Limitation	Explanation
Not optimal	may not find shortest path
Not complete	may get stuck in infinite path
Sensitive to path order	wrong path → delay

7. Applications of DFS

Application	Example
Puzzle solving	backtracking problems
Graph traversal	cycle detection
AI search problems	state space exploration
Maze solving	path finding

8. DFS vs BFS

Feature	DFS	BFS
Strategy	depth-wise	level-wise
Data Structure	stack	queue
Memory	low	high
Optimal	no	yes
Speed	fast (deep)	fast (shallow)

Real-Life Example

Maze solving:

- DFS goes deep into one path
- if dead end → backtracks
- tries another path

Uniform Cost Search (UCS)

Uniform Cost Search (UCS) is an **uninformed search algorithm** that finds the **lowest-cost path** from the start node to the goal node.

Unlike BFS (which minimizes number of steps), UCS considers the **actual cost of each path** and always expands the node with the **smallest cumulative cost**.

Overview

Concept	Meaning
UCS	search based on path cost
Search Type	Uninformed search
Data Structure	Priority Queue
Goal	find minimum-cost path

1. Basic Idea of UCS

Always expand the node with lowest path cost

UCS selects the node with **minimum cumulative cost** ($g(n)$).

Key Formula

$g(n)$ = cost from start node to current node

2. How UCS Works

UCS uses a **priority queue** where nodes are sorted by their cost.

Steps of UCS

Step	Description
1	start with initial node (cost = 0)
2	insert into priority queue
3	remove node with lowest cost
4	check if goal
5	expand node and add children with updated cost
6	repeat until goal found

Example

Graph:

A --1--> B --3--> D

A --4--> C --1--> D

Step-by-step:

Path	Cost
A → B → D	1 + 3 = 4
A → C → D	4 + 1 = 5

UCS selects:

A → B → D (cost = 4)

because it is the **lowest cost path**.

3. UCS Algorithm (Simple)

UCS(start):

1. create priority queue
2. insert start node with cost = 0
3. while queue is not empty:
 node = remove lowest cost node

if node is goal:
 return solution

for each child:
 calculate cost
 add to queue

4. Properties of UCS

Property	Value
Completeness	Yes
Optimality	Yes (if costs are positive)
Time Complexity	$O(b^{(1 + \lfloor C^*/\epsilon \rfloor)})$
Space Complexity	High

Where:

- C^* = cost of optimal solution
- ϵ = minimum step cost

5. Advantages of UCS

Advantage	Explanation
Optimal solution	finds least-cost path
Complete	always finds solution
Handles variable costs	better than BFS

6. Limitations of UCS

Limitation	Explanation
Slow	explores many nodes
High memory usage	stores all nodes
Not efficient	if costs are large

7. UCS vs BFS

Feature	UCS	BFS
Criteria	path cost	number of steps
Data Structure	priority queue	queue
Optimal	yes (cost-based)	yes (step-based)
Use case	weighted graph	unweighted graph

8. Real-Life Example

Google Maps Navigation:

- Roads have different distances
- UCS finds the **shortest distance route**

9. When to Use UCS

Use UCS when:

- path costs are different
- you need **minimum cost solution**
- graph is weighted

Uniform Cost Search is an important algorithm because it **extends BFS to handle real-world problems where costs vary**.

A* Search (A-Star)

A* is an **informed (heuristic) search algorithm** used to find the **optimal path with minimum cost**. It combines:

- actual cost from start $\rightarrow g(n)$
- estimated cost to goal $\rightarrow h(n)$

A* selects the node with the **lowest total estimated cost**.

Overview

Concept	Meaning
A* Search	heuristic-based optimal search
Search Type	Informed search
Data Structure	Priority Queue
Goal	find lowest-cost path efficiently

1. Key Idea of A*

Choose node with minimum $f(n)$

Evaluation Function

$$f(n) = g(n) + h(n)$$

Where:

Term	Meaning
$g(n)$	cost from start to node
$h(n)$	estimated cost to goal (heuristic)
$f(n)$	total estimated cost

2. How A* Works

A* always selects the node with **smallest $f(n)$** from the priority queue.

Steps of A*

Step	Description
1	start with initial node
2	calculate $f(n) = g(n) + h(n)$
3	add to priority queue
4	select node with lowest $f(n)$
5	expand node and update costs
6	repeat until goal reached

3. Example

Graph:

A --1--> B --3--> D

A --4--> C --1--> D

Heuristic values:

Node	$h(n)$
A	5
B	3
C	2

D	0
---	---

Calculation

Path	g(n)	h(n)	f(n)
A → B	1	3	4
A → C	4	2	6

A* chooses:

A → B

Then:

A → B → D = cost 4

4. A* Algorithm (Simple)

A*(start):

1. open list = priority queue
2. add start node
3. while open list not empty:
node = node with lowest f(n)

if node is goal:
return solution

expand node
update g(n), h(n), f(n)
add children to queue

5. Properties of A*

Property	Value
Completeness	Yes
Optimality	Yes (if heuristic is admissible)
Time Complexity	exponential (worst case)
Space Complexity	high

6. Heuristic Function

A heuristic must satisfy:

Admissible Heuristic

$h(n) \leq \text{actual cost}$

This ensures A* finds **optimal solution**.

Example Heuristic

Problem	Heuristic
Map navigation	straight-line distance
Puzzle	number of misplaced tiles

7. Advantages of A*

Advantage	Explanation
Optimal solution	finds best path

Efficient	explores fewer nodes than UCS
Flexible	works with heuristics

8. Limitations of A*

Limitation	Explanation
High memory usage	stores many nodes
Depends on heuristic	poor heuristic → slow
Complex	harder than BFS/DFS

9. A* vs UCS

Feature	A*	UCS
Knowledge	uses heuristic	no heuristic
Speed	faster	slower
Efficiency	high	low
Optimal	yes	yes

10. Real-Life Applications

Application	Example
GPS Navigation	shortest path
Game AI	pathfinding
Robotics	motion planning
Maps (Google, Uber)	route optimization

11. Simple Intuition

Scenario	Explanation
UCS	explores all paths
A*	goes toward goal intelligently

Summary

Concept	Meaning
A*	heuristic-based optimal search
f(n)	$g(n) + h(n)$
Goal	minimum-cost path
Use	pathfinding and AI

A* is one of the **most important algorithms in AI**, combining:

- optimality of UCS
- speed of heuristic search

Greedy Best-First Search (GBFS)

Greedy Best-First Search is an **informed (heuristic) search algorithm** that always expands the node that **appears closest to the goal** according to a heuristic.

It is called *greedy* because it **chooses the locally best option at each step** without considering the total path cost.

Overview

Concept	Meaning
Greedy Best-First Search	heuristic-based search

Search Type	Informed search
Evaluation Function	uses only heuristic
Goal	reach goal quickly

1. Key Idea

Choose node with smallest $h(n)$

Where:

Term	Meaning
$h(n)$	estimated cost from node to goal

Unlike A^* , it **ignores $g(n)$** (actual cost so far).

Evaluation Function

$$f(n) = h(n)$$

2. How It Works

The algorithm uses a **priority queue** and selects the node with **minimum heuristic value**.

Steps

Step	Description
1	start with initial node
2	calculate heuristic $h(n)$
3	add node to priority queue
4	select node with smallest $h(n)$
5	expand node and add children
6	repeat until goal found

3. Example

Graph:

$A \xrightarrow{1} B \xrightarrow{3} D$

$A \xrightarrow{4} C \xrightarrow{1} D$

Heuristic values:

Node	$h(n)$
A	5
B	3
C	2
D	0

Decision

Greedy chooses: $A \rightarrow C$ (because $h(C) = 2$ is smallest)

Then: $A \rightarrow C \rightarrow D$

Important Observation

Path cost:

Path	Cost
$A \rightarrow B \rightarrow D$	4 (better)
$A \rightarrow C \rightarrow D$	5

Greedy chooses **wrong path** because it only looks at $h(n)$.

4. Algorithm (Simple)

Greedy(start):

1. create priority queue
2. insert start node
3. while queue not empty:
 node = node with lowest $h(n)$

if node is goal:
 return solution

expand node
add children to queue

5. Properties

Property	Value
Completeness	No (may fail in infinite graphs)
Optimality	No
Time Complexity	$O(b^m)$
Space Complexity	$O(b^m)$

Where:

- b = branching factor
- m = maximum depth

6. Advantages

Advantage	Explanation
Fast	reaches goal quickly
Simple	easy to implement
Efficient	fewer nodes explored

7. Limitations

Limitation	Explanation
Not optimal	may give wrong path
Incomplete	may get stuck
Depends on heuristic	poor heuristic = bad result

8. Greedy vs A*

Feature	Greedy	A*
Function	$f(n) = h(n)$	$f(n) = g(n) + h(n)$
Optimal	No	Yes
Speed	Faster	Slightly slower
Accuracy	Low	High

9. Real-Life Example

Finding nearest restaurant:

- Greedy picks the closest visible one
- but may not be the shortest route overall

10. When to Use

Use Greedy when:

- speed is more important than accuracy
- approximate solution is acceptable
- heuristic is strong

Dr. Rup Patel/16120

2.5 Generate-and-Test Method

This is one of the **simplest AI search techniques** and forms the basis for many other algorithms.

Generate and Test : Generate and Test is one of the **simplest problem-solving techniques in Artificial Intelligence**.

It works on a **trial-and-error approach**, where the system generates possible solutions and tests each one to see if it satisfies the goal.

The method continues until a **correct solution is found**.

This approach is also known as **brute-force search** because it checks many possibilities.

1. Concept of Generate and Test

Meaning : The **Generate and Test method** involves two main steps:

1. **Generate** possible solutions.
2. **Test** each solution to check whether it satisfies the goal condition.

If the generated solution does not satisfy the goal, the system generates another possible solution and tests again.

Basic Idea

Generate → Test → If goal reached → Stop

↓

Otherwise generate another solution

This process repeats until:

- a solution is found, or
- all possibilities are exhausted.

Example

Suppose we want to find a number whose square equals 25.

Possible numbers:

$$1^2 = 1$$

$$2^2 = 4$$

$$3^2 = 9$$

$$4^2 = 16$$

$$5^2 = 25$$

When 5 is tested, the goal condition is satisfied.

Therefore: **5 is the solution**.

2. Steps of Generate and Test Algorithm

The algorithm follows a simple sequence.

Step	Description
Step 1	Generate a possible solution
Step 2	Test whether it satisfies the goal
Step 3	If yes → return solution
Step 4	If no → generate another solution
Step 5	Repeat until solution is found or search space exhausted

Pseudo Algorithm

1. Generate a possible solution
2. Test the generated solution
3. If solution satisfies goal

- return solution
- 4. Else
 - generate another solution
- 5. Repeat until solution found

Example: 8-Puzzle Problem

Initial state:

1 2 3
4 5 6
7 _ 8

Goal state:

1 2 3
4 5 6
7 8 _

Generate possible moves:

- move blank right
- move blank left
- move blank up

Test each generated state.

When the goal configuration appears, the solution is found.

Example: Password Cracking

Generate possible passwords:

aaa
aab
aac
...
abc

Each password is tested until the correct one is found.

This is a real-world example of generate-and-test.

Types of Generate and Test

There are two ways solutions can be generated.

Method	Description
Random Generation	Solutions generated randomly
Systematic Generation	Solutions generated in structured order

Systematic generation is usually more efficient.

3. Advantages of Generate and Test

Advantage	Explanation
Simple concept	easy to understand and implement
General method	applicable to many problems
No complex knowledge required	works even with minimal information

Example Advantages

Useful in problems like:

- puzzle solving
- mathematical problems

- simple search tasks

4. Limitations of Generate and Test

Despite its simplicity, the method has several drawbacks.

Limitation	Explanation
Inefficient for large problems	search space may be huge
High computation time	many solutions must be tested
No guarantee of optimal solution	may not find best solution
Blind search	no guidance toward goal

Example of Limitation

Consider **chess**.

Possible moves are extremely large.

Testing every possible move using generate and test would be impractical.

Therefore more advanced techniques like:

- heuristic search
- alpha-beta pruning

are used.

Relationship with Other AI Search Methods

Generate and Test forms the **basic idea behind many search algorithms**.

Many advanced algorithms improve this approach by:

- reducing unnecessary solutions
- guiding the search toward promising states

Examples:

Algorithm	Improvement
Hill Climbing	chooses better states
Best First Search	uses heuristics
A* Algorithm	optimal heuristic search

Real-World Applications

Application	Use
Puzzle solving	testing configurations
Cryptography	password guessing
Game playing	testing possible moves
Planning systems	evaluating possible plans

Summary

Concept	Explanation
Generate	produce possible solutions
Test	check if solution satisfies goal
Approach	trial and error
Advantage	simple and general
Limitation	inefficient for large problems

Generate and Test is the **foundation of many AI search techniques**, though more advanced methods are usually required for complex problems.

Hill Climbing

Hill Climbing is a **heuristic search algorithm** used in Artificial Intelligence to solve optimization problems.

It is based on the idea of **gradually improving the current solution by moving toward better states**.

The algorithm compares neighboring states and chooses the one that **improves the evaluation value**.

The name comes from the idea of **climbing a hill toward the highest peak**, where the peak represents the best solution.

Basic Idea of Hill Climbing

Imagine standing on a hill and trying to reach the **highest point**.

Strategy:

1. Look at nearby positions
2. Move to the position that is higher
3. Repeat the process
4. Stop when no better position is available

Conceptual Representation

Goal (Peak)



Start Local Maximum

The algorithm tries to reach the **highest peak**, but sometimes it may get stuck at smaller peaks.

Characteristics of Hill Climbing

Feature	Description
Local search method	examines neighboring states
Greedy approach	always moves toward better state
Memory efficient	does not store full search tree
Heuristic based	uses evaluation function

Evaluation Function

Hill climbing uses an **evaluation function** to measure how good a state is.

Example:

$f(\text{state}) = \text{quality of the solution}$

Higher value $\rightarrow$ better state.

The algorithm always chooses the state with the **highest evaluation value**.

Example

Suppose we want to maximize the function:

$$f(x) = -(x^2) + 6x$$

Starting value:

$$x = 1$$

Check neighbors:

$$x = 2$$

$$x = 3$$

$$x = 4$$

Choose the value that increases the function the most.

Continue until the best value is reached.

Types of Hill Climbing

There are two major types:

Type	Description
Simple Hill Climbing	checks neighbors sequentially
Steepest Ascent Hill Climbing	evaluates all neighbors and selects best

1. Simple Hill Climbing

Meaning : In **Simple Hill Climbing**, the algorithm checks neighbors **one by one** and moves to the first better state it finds.

If no better neighbor is found, the algorithm stops.

Algorithm Steps

1. Start with an initial state.
2. Generate neighboring states.
3. Compare neighbor with current state.
4. If neighbor is better → move to that state.
5. Repeat until no better state exists.

Pseudo Algorithm

1. Start with initial state
2. Generate a neighbor
3. If neighbor is better
 move to neighbor
4. Repeat until no improvement

Example

Suppose the evaluation values are:

State	Value
A	5
B	7
C	6

If the algorithm moves from A to B (first better neighbor), it stops searching further neighbors.

Advantages

- Simple to implement
- Low memory usage

Limitation : It may miss a **better state** because it stops at the first improvement.

2. Steepest Ascent Hill Climbing

Meaning: In **Steepest Ascent Hill Climbing**, the algorithm examines **all neighboring states** and chooses the one with the **maximum improvement**.

This ensures the algorithm always moves in the **best possible direction**.

Algorithm Steps

1. Start with initial state
2. Generate all neighboring states
3. Evaluate each neighbor

4. Select the best neighbor
5. Move to that state
6. Repeat until no improvement is possible

Example

Suppose we have:

State	Value
Current State	5
Neighbor 1	6
Neighbor 2	9
Neighbor 3	7

The algorithm selects **Neighbor 2 (value 9)** because it is the best.

Advantages

- Better search direction
- Finds higher quality solutions

Limitation

- Requires more computation
- Must evaluate all neighbors

Problems of Hill Climbing

Hill climbing has several limitations.

The algorithm may fail because of the following problems:

1. Local Maximum
2. Plateau
3. Ridge

1. Local Maximum

Meaning: A **Local Maximum** is a state that is better than its neighbors but **not the best possible solution globally**.

The algorithm stops because no better neighbor exists.

Start

The algorithm reaches the local peak and stops, even though a higher peak exists.

Example

Imagine climbing hills: You reach a small hilltop but the real mountain is somewhere else.

2. Plateau

Meaning: A **Plateau** is a flat region where all neighboring states have the **same value**.

The algorithm cannot decide which direction to move.

Example

If evaluation values are:

State	Value
A	5
B	5
C	5

The algorithm cannot determine which state is better.

3. Ridge

Meaning: A **Ridge** is a situation where the optimal path requires moving in **multiple directions simultaneously**, but hill climbing only moves in one direction.

This makes it difficult for the algorithm to climb the ridge.

Solutions to Hill Climbing Problems

Several improvements are used to overcome these issues.

Method	Idea
Random Restart Hill Climbing	start search from multiple points
Simulated Annealing	allow occasional downhill moves
Stochastic Hill Climbing	choose random better neighbor

Applications of Hill Climbing

Hill climbing is widely used in optimization problems.

Application	Example
Game playing	move selection
Machine learning	parameter optimization
Robotics	path optimization
Scheduling	resource allocation

Comparison of Hill Climbing Types

Feature	Simple Hill Climbing	Steepest Ascent Hill Climbing
Neighbor checking	one by one	all neighbors
Speed	faster	slower
Accuracy	lower	higher
Computation	low	high

Summary

Concept	Meaning
Hill Climbing	heuristic optimization search
Simple Hill Climbing	first better neighbor selected
Steepest Ascent	best neighbor selected
Local Maximum	suboptimal peak
Plateau	flat search region
Ridge	difficult search path

Hill Climbing is a **fundamental optimization algorithm in AI**, but it may get stuck due to local search limitations.

3. Priority Queue

Meaning: Best-First Search uses a **Priority Queue** to decide which node should be expanded next.

A priority queue is a data structure where elements are removed according to **priority value instead of order of insertion**.

Priority Queue Operation

Nodes are stored along with their evaluation values.

The node with the **best evaluation value** is selected first.

Example

Suppose the queue contains nodes:

Node	f(n) Value
A	9
B	5
C	7

The priority queue removes:

$B \rightarrow C \rightarrow A$

because $5 < 7 < 9$.

Best-First Search Algorithm

Steps

1. Place the initial node in the priority queue.
2. Evaluate it using the heuristic function.
3. Remove the node with the best evaluation value.
4. Generate its successor nodes.
5. Insert successors into the priority queue.
6. Repeat until the goal node is found.

Pseudo Algorithm

1. OPEN $\leftarrow$ priority queue containing initial state
2. LOOP
3. remove best node from OPEN
4. if node is goal $\rightarrow$ success
5. generate successors
6. evaluate successors
7. insert successors into OPEN
8. END LOOP

Example: Route Finding

Goal: reach City G

Heuristic values:

City	h(n)
A	10
B	7
C	4
D	6
G	0

Search steps:

1. Start at A
2. Expand neighbors B and C
3. Choose C
4. Expand $C \rightarrow G$
5. Goal reached

The algorithm always selects the **most promising node**.

Comparison with Other Search Algorithms

Algorithm	Uses Heuristic	Optimal
Breadth First Search	No	Yes (for equal cost)
Hill Climbing	Yes	Not guaranteed
Best-First Search	Yes	Not guaranteed
A* Search	Yes	Optimal

Applications of Best-First Search

Application	Example
Pathfinding	navigation systems
Game playing	selecting best move
Robotics	route planning
AI planning	task planning systems

Summary

Concept	Explanation
Best-First Search	heuristic-based search algorithm
Evaluation Function	measures how good a state is
Greedy Search	selects node closest to goal
Priority Queue	selects node with best evaluation

Best-First Search improves search efficiency by **using heuristics to guide exploration toward promising solutions**.

2.8 Problem Reduction

Key concepts include:

- Breaking large problems into smaller subproblems
- AND-OR graphs
- Decomposition strategies.

2.8 Problem Reduction: Problem Reduction is an AI problem-solving technique where a **complex problem is broken into smaller, simpler subproblems**. Each subproblem is solved individually, and their solutions are combined to solve the original problem.

Instead of solving the entire problem directly, the system **reduces the problem step by step** until it reaches problems that are easy to solve.

This approach is commonly used in:

- planning systems
- theorem proving
- natural language processing
- complex decision making

Basic Idea of Problem Reduction

The idea is similar to **divide and conquer**.

Steps:

1. Identify a complex problem.
2. Divide it into smaller subproblems.
3. Solve each subproblem.
4. Combine the results to obtain the final solution.

Conceptual Representation

Original Problem

|

v

Subproblem 1

|

v

Subproblem 2

|

v

Smaller Problems

|

v

Solution

Breaking problems into smaller parts makes the problem **easier and more manageable**.

Example of Problem Reduction

Problem: Preparing a Meal

Original task: Prepare dinner.

Subproblems:

1. Cook rice
2. Prepare vegetables
3. Cook curry

Each of these tasks can further be divided into smaller tasks.

Another Example: Solving a Mathematical Expression

Problem:

$$(2 + 3) \times (4 + 5)$$

Subproblems:

1. Solve $(2 + 3)$
2. Solve $(4 + 5)$
3. Multiply results

Solution: $5 \times 9 = 45$

Problem Decomposition

Meaning: Problem Decomposition is the process of dividing a large problem into smaller subproblems that can be solved independently.

This is a fundamental concept in AI problem solving.

Why Problem Decomposition is Important

Complex problems often contain:

- many variables
- multiple steps
- large search spaces

Breaking them into smaller problems helps:

- reduce complexity
- simplify reasoning
- improve efficiency

Example: Travel Planning

Goal: Travel from City A to City D.

Possible decomposition:

Step	Subproblem
1	Travel A → B
2	Travel B → C
3	Travel C → D

Solving each step helps achieve the final goal.

Types of Problem Decomposition

Type	Description
Sequential decomposition	subproblems solved in sequence
Parallel decomposition	subproblems solved independently
Hierarchical decomposition	problems divided into multiple levels

AND-OR Graphs

Problem reduction is often represented using **AND-OR graphs**.

These graphs describe how problems can be broken into subproblems.

What is an AND-OR Graph?

An **AND-OR graph** is a graphical representation used to show how a problem can be solved by solving smaller subproblems.

The graph contains two types of nodes:

- AND nodes
- OR nodes

Components of AND-OR Graph

Component	Meaning
Node	represents a problem or subproblem
Edge	represents relationship between problems
AND node	all subproblems must be solved
OR node	only one subproblem must be solved

OR Node : An **OR node** means that **only one of the possible paths needs to be solved**.

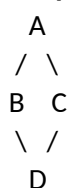
Example: To travel from City A to City D:

Possible routes:

- A → B → D
- A → C → D

Only one path is needed.

OR Graph Representation



Either path through B or C leads to the goal.

AND Node

An **AND node** means **all subproblems must be solved**.

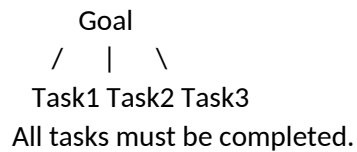
Example: To prepare tea:

Tasks:

1. Boil water
2. Add tea leaves
3. Add sugar

All steps are required.

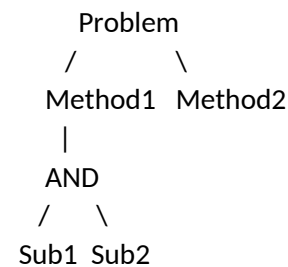
AND Graph Representation



Combined AND-OR Graph

Many problems contain both AND OR relationships.

Example:



Meaning:

- Choose **Method1 OR Method2**
- If Method1 is selected, **both Sub1 and Sub2 must be solved**

Example: AI Planning Problem

Goal: Travel from home to university.

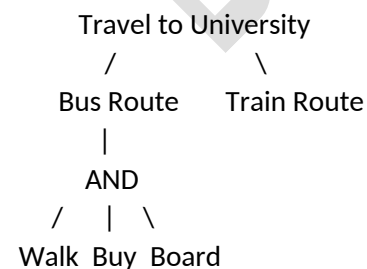
Options:

1. Bus route
2. Train route

If bus route chosen:

- walk to bus stop
- buy ticket
- board bus

Representation:



Advantages of Problem Reduction

Advantage	Explanation
Reduces complexity	breaks large problems into smaller ones
Easier problem solving	smaller tasks easier to solve
Modular approach	independent subproblem solving
Efficient search	reduces search space

Limitations

Limitation	Explanation
decomposition may be difficult	not all problems divide easily
subproblem interaction	solutions may depend on each other
increased overhead	managing many subproblems

Applications of Problem Reduction

Application	Example
Planning systems	robot task planning
Natural language processing	sentence parsing
Game playing	strategy decomposition
Theorem proving	breaking complex proofs

Comparison: State Space vs Problem Reduction

Approach	Idea
State Space Search	explore possible states
Problem Reduction	divide problem into subproblems

Both approaches are important in AI problem solving.

Summary

Concept	Meaning
Problem Reduction	breaking complex problems into smaller parts
Problem Decomposition	dividing problem into manageable subproblems
AND Node	all subproblems must be solved
OR Node	any one subproblem is sufficient
AND-OR Graph	graphical representation of problem reduction

Problem reduction helps AI systems **solve complex tasks more efficiently by organizing problems into smaller solvable components.**

2.9 Constraint Satisfaction Problems (CSP)

This topic includes:

- variables
- domains
- constraints
- constraint propagation
- real-world CSP examples like Sudoku and scheduling.

Constraint Satisfaction Problems (CSP): A **Constraint Satisfaction Problem (CSP)** is a type of problem in Artificial Intelligence where the goal is to **find values for variables such that all given constraints are satisfied.**

Instead of searching blindly through all possible solutions, CSP focuses on **satisfying restrictions (constraints)** that limit possible solutions.

CSPs are widely used in:

- scheduling
- planning
- resource allocation
- puzzle solving (Sudoku, N-Queens)

Basic Idea of CSP

A CSP problem has three main elements:

1. Variables
2. Domains
3. Constraints

The goal is to assign values to variables **so that no constraint is violated**.

Example

Suppose three people must be assigned different colors.

Variables: Person1, Person2, Person3

Domain (possible colors): {Red, Green, Blue}

Constraint: No two persons can have the same color

The solution must satisfy this constraint.

Components of Constraint Satisfaction Problem

Component	Description
Variables	elements whose values must be determined
Domains	possible values each variable can take
Constraints	restrictions on variable assignments

1. Variables

Meaning: Variables are the **unknown elements in the problem that must be assigned values**.

Each variable represents a **decision that needs to be made**.

Example

In a **Sudoku puzzle**:

Variables: Cell1, Cell2, Cell3 ... Cell81

Each cell is a variable.

Example

In scheduling:

Variables may represent:

Course1

Course2

Course3

Each variable must be assigned a time slot.

2. Domains

Meaning : The **domain** of a variable is the **set of possible values that the variable can take**.

Each variable has its own domain.

Example

Sudoku puzzle:

Domain for each cell: {1,2,3,4,5,6,7,8,9}

Example

Scheduling problem:

Domain for each course: {Monday, Tuesday, Wednesday}

The algorithm must choose values from the domain.

3. Constraints

Meaning : Constraints are **rules that restrict which combinations of values are allowed**.

They define relationships between variables.

Types of Constraints

Type	Description
Unary constraint	affects one variable
Binary constraint	involves two variables
Global constraint	involves many variables

Unary Constraint

A constraint applied to a **single variable**.

Example: Course1 cannot be scheduled on Monday

Binary Constraint

A constraint involving **two variables**.

Example: Course1 and Course2 cannot be at the same time

Global Constraint

A constraint involving **multiple variables**.

Example: All Sudoku numbers in a row must be different

Example: Map Coloring Problem

Goal: Color a map so that neighboring regions have different colors.

Variables:

Regions A, B, C

Domain: {Red, Green, Blue}

Constraints:

$A \neq B$

$B \neq C$

$A \neq C$

The algorithm must assign colors satisfying these constraints.

Constraint Propagation

Meaning : **Constraint propagation** is a technique used to **reduce the number of possible values in domains by applying constraints early**.

It helps eliminate impossible values before the search begins.

This significantly reduces the **search space**.

How Constraint Propagation Works

When a variable receives a value, constraints remove inconsistent values from other variables.

Example:

Variables: A, B

Domain:

$A = \{1,2,3\}$

$B = \{1,2,3\}$

Constraint:

$A \neq B$

If: $A = 1$

Then remove **1** from B's domain.

New domain: $B = \{2,3\}$

Example: Sudoku

Constraint propagation helps eliminate impossible numbers.

Example:

Row already contains: 1,2,3,4

Remaining domain for empty cell: {5,6,7,8,9}

Values **1-4** are **removed** from the domain.

This simplifies the problem.

Benefits of Constraint Propagation

Benefit	Explanation
Reduces search space	fewer possibilities to check
Faster solution	eliminates invalid assignments early
Improves efficiency	reduces computation time

Real-World CSP Examples

Problem	Variables	Constraints
Sudoku	grid cells	numbers cannot repeat
N-Queens	queen positions	queens cannot attack each other
Scheduling	time slots	no overlapping tasks
Map coloring	regions	adjacent regions different color

Example: N-Queens Problem

Goal: Place **N queens on chessboard** so that none attack each other.

Variables: Queen positions

Domain: board positions

Constraints:

- queens cannot share same row
- queens cannot share same column
- queens cannot share same diagonal

Why CSP is Important in AI

CSP techniques help solve many complex problems by:

- reducing unnecessary search
- applying constraints early
- improving efficiency of search algorithms

Many modern AI systems use CSP techniques in:

- planning
- robotics
- scheduling systems

- resource management

Summary

Concept	Meaning
Constraint Satisfaction Problem	find values satisfying all constraints
Variables	unknown elements to assign
Domains	possible values for variables
Constraints	rules restricting assignments
Constraint Propagation	reducing domains using constraints

CSP is an important AI technique because it **transforms complex problems into structured problems that can be solved systematically.**

2.10 Means-Ends Analysis

This is a problem-solving technique used in **planning and reasoning systems.**

Means-Ends Analysis: Means-Ends Analysis (MEA) is a problem-solving technique used in Artificial Intelligence where the system tries to **reduce the difference between the current state and the goal state.**

The method works by:

1. Comparing the current situation with the desired goal
2. Identifying the differences
3. Selecting actions (operators) that reduce those differences

The process continues until the **difference becomes zero**, meaning the goal is achieved.

This technique was used in early AI programs like the **General Problem Solver (GPS).**

Basic Idea of Means-Ends Analysis

The idea is simple:

Current State → Compare with Goal → Identify Difference → Apply Operator → Reduce Difference

The algorithm repeatedly **reduces the gap between the present situation and the desired goal.**

Example

Suppose you want to travel from **City A to City D.**

Current state: Location = City A

Goal state: Location = City D

Difference: Distance between A and D

Operator: Travel from A → B → C → D

Each step reduces the difference until the goal is reached.

Main Components of Means-Ends Analysis

The technique mainly involves three concepts:

Component	Meaning
Goal Comparison	comparing current state with goal state
Difference Reduction	identifying and reducing differences
Operator Selection	choosing action to reduce difference

1. Goal Comparison

Meaning : Goal comparison is the process of **comparing the current state with the desired goal state.**

The system analyzes:

- what is the current situation
- what the goal situation should be
- how different they are

This step identifies the **gap between the present state and the goal**.

Example

Goal: temperature of room = 22°C

Current state: Temperature = 28°C

Difference: 6°C higher than desired

The system recognizes that the temperature must be reduced.

2. Difference Reduction

Meaning : Difference reduction means **finding ways to reduce the difference between the current state and the goal state**.

The system identifies **what changes must occur to achieve the goal**.

Example

Current state: Room temperature = 28°C

Goal: Room temperature = 22°C

Difference: Temperature difference = 6°C

Possible solution: Turn on air conditioner

This action reduces the difference.

Another Example

Goal: solve equation

$x + 5 = 12$

Difference: 5 added to x

Action: Subtract 5 from both sides

Result: $x = 7$

Difference reduced to zero.

3. Operator Selection

Meaning : Operators are **actions that transform one state into another state**.

Operator selection involves choosing **the best action that reduces the difference between current state and goal state**.

Example

Goal: reach destination

Possible operators:

Operator	Action
Drive car	travel by road
Take train	travel by railway
Fly	travel by airplane

The system selects the operator that **best reduces the difference between current location and destination**.

Means-Ends Analysis Process

The process follows a repeated cycle.

Step	Description
------	-------------

Step 1	Compare current state with goal state
Step 2	Identify differences
Step 3	Select operator to reduce difference
Step 4	Apply operator
Step 5	Update current state
Step 6	Repeat until goal reached

Algorithm Representation

1. Compare current state with goal state
2. Identify the differences
3. Select an operator that reduces the difference
4. Apply the operator
5. Update the current state
6. Repeat until the goal is achieved

Example: Making Tea

Goal: Tea prepared

Current state: No tea prepared

Difference: Ingredients not prepared

Operators:

1. Boil water
2. Add tea leaves
3. Add sugar
4. Add milk

Each action reduces the difference between current state and goal.

Example: Puzzle Problem

Goal state:

1 2 3

4 5 6

7 8 _

Current state:

1 2 3

4 5 6

7 _ 8

Difference:

8 and blank positions incorrect

Operator: Move blank right

Goal achieved.

Advantages of Means-Ends Analysis

Advantage	Explanation
Reduces search space	focuses on reducing differences
Goal-directed search	search guided toward goal
Efficient planning	avoids unnecessary exploration

Limitations

Limitation	Explanation
------------	-------------

Requires good difference identification	difficult for complex problems
May require subgoals	intermediate steps needed
Operator selection may be complex	multiple operators possible

Applications of Means-Ends Analysis

Field	Application
Robotics	planning robot actions
Automated planning	scheduling tasks
Problem solving systems	reasoning systems
Theorem proving	step-by-step proof construction

Relationship with State Space Search

Approach	Idea
State Space Search	explore all possible states
Means-Ends Analysis	focus on reducing difference toward goal

Means-Ends Analysis is more **goal-directed**, which often reduces search effort.

Summary

Concept	Meaning
Means-Ends Analysis	problem solving by reducing differences
Goal Comparison	compare current and goal states
Difference Reduction	identify changes needed
Operator Selection	choose action that reduces difference

Means-Ends Analysis helps AI systems **solve problems by systematically moving from the current state toward the goal state.**